

Comprehensive Guide to Sorting Algorithms

Sorting algorithms are fundamental building blocks in computer science that rearrange elements in a specific order. Mastering their time and space complexities, stability, and adaptive properties is crucial for algorithm design and data structure analysis exams.

Classification of Sorting Algorithms

- Comparison-based sorts determine relative order using comparison operators, bound by a lower limit of $\mathcal{O}(n \log n)$.
- Non-comparison sorts (Counting, Radix, Bucket) exploit specific data properties to sort in linear time $\mathcal{O}(n)$.
- In-place algorithms require a constant $\mathcal{O}(1)$ auxiliary memory space beyond the input array.
- Stable sorting algorithms preserve the relative order of records with equal keys.

$$T(n) = 2T\left(\frac{n}{2}\right) + \mathcal{O}(n)$$

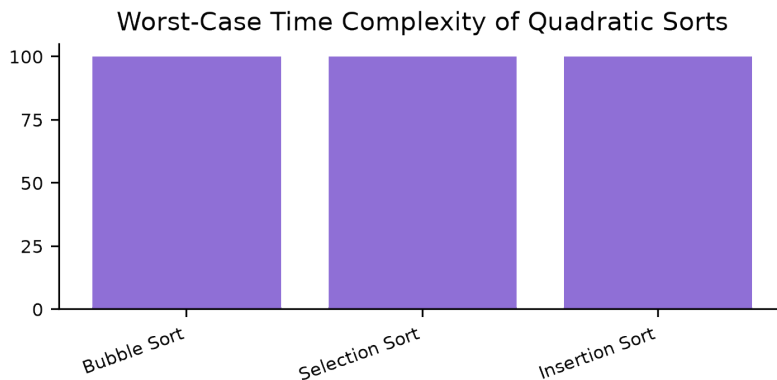
Sorting Algorithm Classification Workflow



Comparison-Based Quadratic Sorts

- Bubble Sort repeatedly swaps adjacent elements if they are in the wrong order; best case $\mathcal{O}(n)$ if optimized, worst case $\mathcal{O}(n^2)$.
- Selection Sort repeatedly finds the minimum element and places it at the beginning; always $\mathcal{O}(n^2)$ comparisons.
- Insertion Sort builds the final sorted array one item at a time; highly efficient for small or nearly sorted data with $\mathcal{O}(n)$ best case.
- Advantage: Extremely simple to implement and requires zero or minimal auxiliary memory space $\mathcal{O}(1)$.

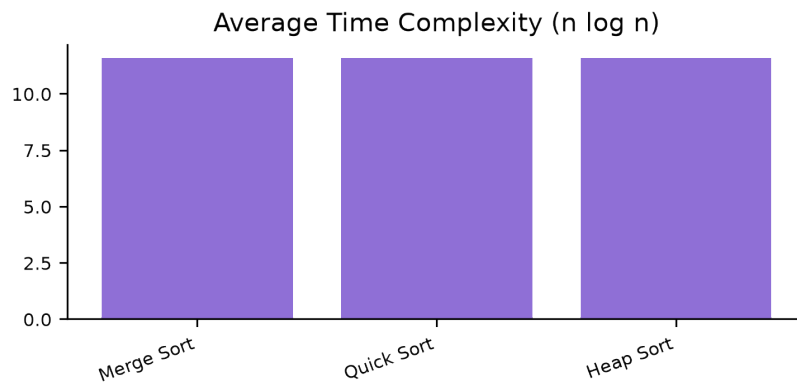
$$S(n) = \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$$



Efficient Divide-and-Conquer Sorts

- Merge Sort splits the array into halves, recursively sorts them, and merges the sorted halves with $\mathcal{O}(n \log n)$ guaranteed time.
- Quick Sort partitions an array around a pivot element; average case is $\mathcal{O}(n \log n)$ but worst case degrades to $\mathcal{O}(n^2)$.
- Heap Sort builds a binary max-heap structure to extract elements systematically in $\mathcal{O}(n \log n)$ time and $\mathcal{O}(1)$ space.
- Disadvantage: Merge sort requires $\mathcal{O}(n)$ extra space, and standard Quick Sort is unstable.

$$T(n) = n \log_2 n$$



Linear-Time Non-Comparison Sorts

- Counting Sort uses an auxiliary frequency array, operating in $\mathcal{O}(n + k)$ time where k is the range of input values.
- Radix Sort processes data digit by digit from least significant to most significant using a stable subroutine like counting sort.
- Bucket Sort distributes elements into uniform buckets which are then individually sorted, typically using insertion sort.
- Advantage: Can bypass the $\mathcal{O}(n \log n)$ lower bound for comparison sorting under restricted key spaces.

$$T(n) = \mathcal{O}(n + k)$$

Algorithmic Comparison Summary

- Use Insertion Sort for small arrays or nearly sorted inputs due to high adaptability.
- Use Merge Sort when guaranteed worst-case performance and stability are mandatory.
- Use Quick Sort for general in-place sorting where average-case speed is prioritized.
- Use Counting/Radix Sort when sorting integers or strings with a known small range.

Key Terms

Stability	A sorting algorithm is stable if it preserves the relative order of records with identical keys.
In-place	An algorithm that transforms input using a constant amount $\mathcal{O}(1)$ of extra memory space.
Adaptability	The property where an algorithm runs faster when the input is already partially sorted.

Pivot	An element chosen in partitioning algorithms around which other elements are rearranged.
Lower Bound	The mathematical limit establishing that any comparison sort must make at least $\log_2(n!)$ comparisons in the worst case.

Exam Tips

- ★ Always memorize the exact best, average, and worst-case time complexities along with auxiliary space for all standard sorts.
- ★ Be prepared to prove or explain why comparison-based sorting cannot perform better than $\mathcal{O}(n \log n)$ using decision trees.
- ★ Remember that Quick Sort's worst case occurs when the chosen pivot is consistently the minimum or maximum element.
- ★ Distinguish clearly between stable algorithms (Merge, Insertion, Bubble, Counting) and unstable ones (Selection, Quick, Heap).